

Universidad de Guadalajara

Centro Universitario de Ciencias Exactas e Ingenierías



División de Tecnologías para la Integración CiberHumana

Ingeniería en Computación

Programación Paralela y Concurrente

2026-A

Proyecto final

Profesor: Zeus Emanuel Gutierrez Cobian

Integrantes del Equipo:

- Jorge Yahir Valenzuela López
- Leonardo Navarro Real
- Alan Imanol Corona Salinas (218413809)

11 de mayo de 2026

Tabla de Contenidos

Tabla de Contenidos	2
Introducción	3
Marco teórico	4
Diseño del sistema.	6
Arquitectura	6
Descripción de componentes	6
Flujos concurrentes.	7
Variables compartidas o colas	7
Problemas potenciales y soluciones	7
Implementación	8
Lenguaje	8
Librerías utilizadas.	8
Explicación breve del código	9
sequential_crawler.py	9
concurrent_crawler.py	9
benchmark.py	9
Capturas de ejecución	10
Evaluación del desempeño	13
Conclusiones	14
Referencias	15

Introducción

Problema seleccionado y justificación

El proyecto consiste en desarrollar un crawler web concurrente capaz de recorrer páginas web, extraer enlaces y procesar información utilizando múltiples hilos. El problema principal es la baja eficiencia de los crawlers secuenciales, ya que procesan una página a la vez y tardan más al manejar grandes cantidades de información.

Este proyecto fue elegido porque la programación concurrente permite mejorar notablemente el rendimiento del sistema al aprovechar mejor los recursos del procesador y reducir tiempos de espera durante las conexiones de red.

Relación con la programación concurrente

El proyecto está directamente relacionado con la programación concurrente porque varias tareas se ejecutan simultáneamente. Mientras un hilo descarga una página, otros pueden analizar enlaces o procesar nuevas URLs.

Esto permite aprovechar mejor el CPU y reducir el tiempo total de ejecución, especialmente en operaciones de red donde existen tiempos de espera constantes.

Objetivo general

Desarrollar un crawler web concurrente capaz de analizar múltiples páginas web simultáneamente mediante técnicas de multithreading y sincronización para optimizar el tiempo de procesamiento.

Objetivos específicos

- Implementar múltiples hilos para procesar páginas web en paralelo.
- Comparar el rendimiento entre la versión secuencial y concurrente.
- Gestionar correctamente la sincronización entre hilos.
- Evitar duplicación de URLs visitadas.
- Aplicar conceptos de programación concurrente en un caso práctico.

Marco teórico

Concurrencia y paralelismo

El proyecto aplica concurrencia y paralelismo para ejecutar múltiples tareas simultáneamente. La concurrencia permite gestionar varias solicitudes web al mismo tiempo, mientras que el paralelismo aprovecha varios hilos para procesar páginas concurrentemente.

Gracias a esto, el crawler concurrente reduce considerablemente el tiempo de ejecución respecto a la versión secuencial.

Hilos / multithreading

El sistema utiliza multithreading mediante la clase `threading.Thread`, donde cada hilo ejecuta la función `worker` para procesar URLs de manera independiente.

Esto permite que varias páginas sean descargadas y analizadas al mismo tiempo, aumentando la eficiencia del crawler.

```
t = threading.Thread(  
    target=worker,  
    args=(task_queue, seed_url, max_pages),  
    name=f"Hilo-{i+1}"  
)
```

Modelo de ejecución empleado

El proyecto utiliza un modelo basado en memoria compartida y productor-consumidor. Los hilos toman URLs desde una cola compartida (`queue.Queue`) y agregan nuevos enlaces encontrados durante el recorrido.

Cada hilo consume tareas y produce nuevas URLs para continuar el proceso de crawling.

Comunicación entre procesos

La comunicación entre hilos se realiza mediante estructuras compartidas como la cola de tareas (`task_queue`) y las listas de URLs visitadas y resultados.

```
task_queue.put(link)  
url = task_queue.get()
```

Cuando un hilo encuentra nuevos enlaces, estos se agregan a la cola global para que otros hilos puedan procesarlos posteriormente.

Sincronización

Debido a que varios hilos acceden a recursos compartidos, el proyecto implementa mecanismos de sincronización utilizando `threading.Lock()`.

Los locks se utilizan para proteger las estructuras compartidas (`visited` y `results`) y evitar condiciones de carrera o inconsistencias en los datos.

```
with visited_lock:
    if url in visited:
        continue
    visited.add(url)
```

Este mecanismo asegura que únicamente un hilo pueda modificar la lista de URLs visitadas al mismo tiempo.

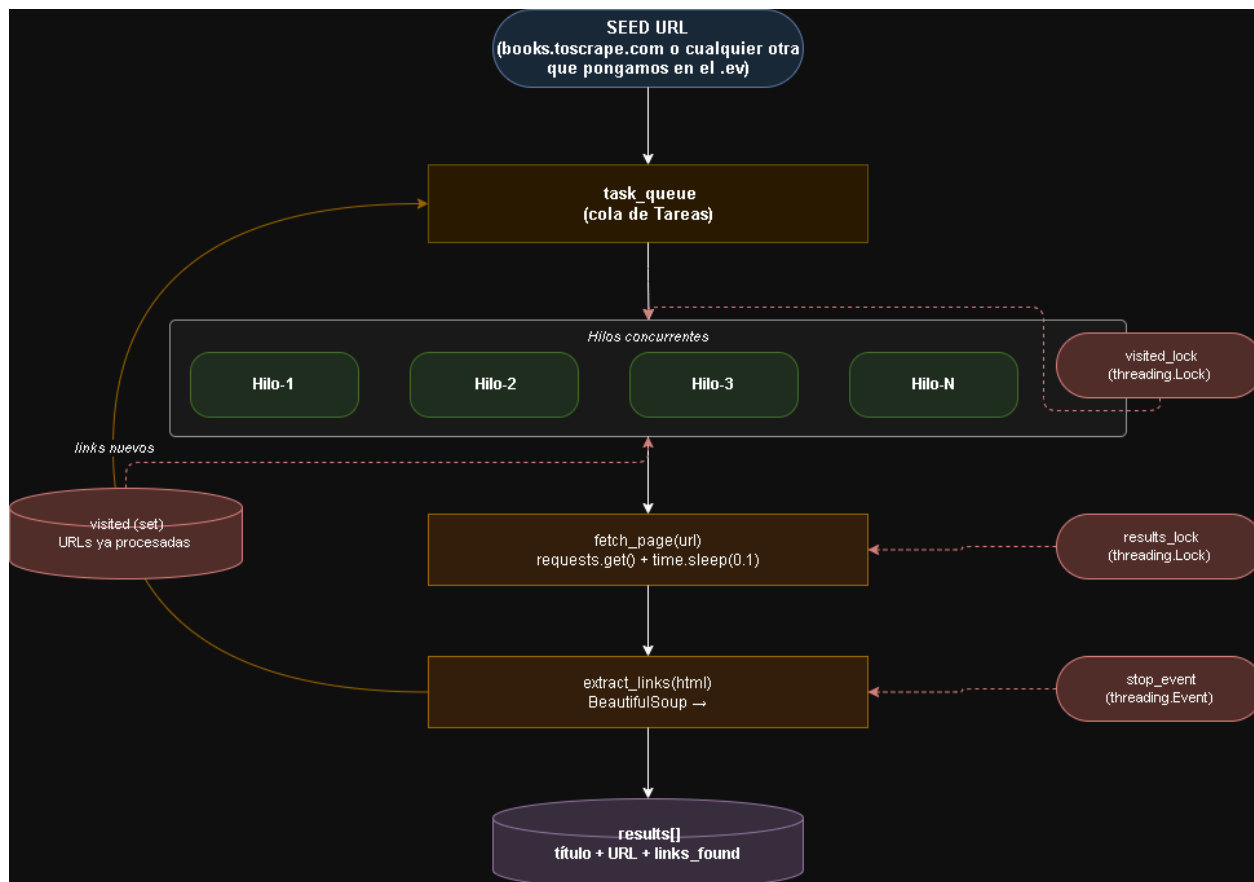
Memoria compartida o distribuida

El proyecto utiliza memoria compartida, ya que todos los hilos trabajan dentro del mismo proceso y acceden a las mismas estructuras de datos compartidas.

Este modelo facilita la comunicación entre hilos y permite compartir información de manera rápida y eficiente.

Diseño del sistema.

Arquitectura



Descripción de componentes

- task_queue: es la cola que almacena las URLs pendientes de visitar
- visited (set): es el registro de URLs ya procesadas, protegida por el visited_lock
- results: es la lista de páginas, almacena títulos y metadatos, se protege por results_lock
- worker: es la función que ejecutan todos los hilos, manda a llamar toda la lógica del programa
- stop_event: cuando ya no hay más cola de tareas, se manda a llamar y detiene todos los hilos, previene deadlocks.

Flujos concurrentes.

El sistema lanza N hilos simultáneamente, todos a ejecutar el worker. Cada hilo sigue el siguiente flujo de manera independiente:

1. Primero toma URL de la cola de pendientes
2. Realiza la descarga del HTML
3. Parsear el HTML
4. Si todo fue exitoso, agrega el link a completados
5. Repite hasta que se acaben los links

Variables compartidas o colas

Variable	Tipo	Protección
task_queue	Queue (cola)	Thread-safe
visited	set	visited_lock
results	list	results_lock
stop_event	threading.Event	Thread-safe

Problemas potenciales y soluciones

- Race condition en visited, dos hilos pueden visitar la misma URL: Para este problema aplicamos un lock llamado 'visited_lock' que protege lectura y escritura en un solo bloque
- Deadlock, los hilos pueden quedarse esperando indefinidamente: Para este problema aplicamos un stop_event, que garantiza que todos los hilos terminen correctamente
- Saturación del servidor: Para este problema precisamente es que sirve la variable de MAX_PAGES, ya que limita el total de peticiones.

Implementación

Lenguaje

Para la realización del Crawler web, utilizamos únicamente el lenguaje de programación de Python, ya que permite trabajar con programación concurrente gracias a su módulo threading y su ecosistema de librerías para procesar la web.

Librerías utilizadas.

```
Python
import requests                # Peticiones HTTP para descargar páginas
web
from bs4 import BeautifulSoup  # Parseo de HTML para extraer datos
from urllib.parse import urljoin  # Convierte URLs relativas en absolutas
from urllib.parse import urlparse  # Extrae componentes de una URL (dominio,
path)
from collections import deque    # Cola eficiente O(1) para URLs pendientes
import threading                # Creación y gestión de hilos concurrentes
import queue                    # Cola thread-safe para distribuir URLs
entre hilos
import time                     # Medición de tiempos y simulación de
latencia
import os                       # Lectura de variables de entorno del
sistema
import matplotlib.pyplot as plt  # Generación de gráficas de resultados
from dotenv import load_dotenv    # Carga variables desde el archivo .env
```


Explicación breve del código

`sequential_crawler.py`

Esta es la versión secuencial del crawler. El archivo se compone de tres funciones auxiliares y una función principal:

- `fetch_page(url)` realiza una petición HTTP GET a la URL indicada y retorna el contenido HTML de la página.
- `extract_links(html, base_url)` parsea el HTML recibido y extrae todos los enlaces presentes, resolviendo las URLs relativas a absolutas mediante `base_url`.
- `extract_title(html)` busca y retorna el texto contenido en la etiqueta `<title>` del HTML.

`crawl_sequential(url, max_pages)` contiene la lógica principal del recorrido. Recibe una URL semilla y el número máximo de páginas a visitar. Internamente mantiene un conjunto de URLs visitadas, una cola de URLs pendientes y una lista de resultados. El recorrido se ejecuta de forma secuencial, procesando una página a la vez hasta alcanzar el límite establecido.

`concurrent_crawler.py`

Esta es la versión concurrente del crawler. Comparte las funciones auxiliares `fetch_page`, `extract_links` y `extract_title` con la versión secuencial, e incorpora dos componentes adicionales:

- `worker()` es la función que ejecuta cada hilo. En un ciclo continuo, toma una URL de la cola compartida, verifica si ya fue visitada, descarga la página y almacena el resultado de forma segura usando Lock.
- `crawl_concurrent(url, max_pages, num_threads)` es la función principal. Lanza N hilos, asignando a cada uno la ejecución de `worker()`. Una vez que la cola de URLs se vacía, espera a que todos los hilos terminen e imprime los resultados obtenidos.

`benchmark.py`

Este archivo orquesta las pruebas de rendimiento. Lee desde el archivo `.env` los parámetros de configuración, como la URL semilla, el número máximo de páginas y las configuraciones de hilos a evaluar (1, 2, 4, 8). Ejecuta primero la versión secuencial como línea base y luego la versión concurrente con cada configuración. Finalmente, calcula el speedup, imprime la tabla comparativa de tiempos y genera una gráfica de resultados que guarda como imagen.

Capturas de ejecución

```
=====
BENCHMARK - CRAWLER WEB CONCURRENTENTE
=====

[2/4] Corriendo versión CONCURRENTENTE con 2 hilo(s)...
[Hilo-1] (1/20) https://books.toscrape.com
[Hilo-1] (2/20) https://books.toscrape.com/catalogue/category/books/historical-fiction_4/index.html
[Hilo-2] (3/20) https://books.toscrape.com/catalogue/category/books/parenting_28/index.html
[Hilo-2] (4/20) https://books.toscrape.com/catalogue/category/books/horror_31/index.html
[Hilo-1] (5/20) https://books.toscrape.com/catalogue/category/books/politics_48/index.html
[Hilo-2] (6/20) https://books.toscrape.com/catalogue/category/books/romance_8/index.html
[Hilo-1] (7/20) https://books.toscrape.com/index.html
[Hilo-1] (8/20) https://books.toscrape.com/catalogue/category/books/new-adult_20/index.html
[Hilo-2] (9/20) https://books.toscrape.com/catalogue/scott-pilgrims-precious-little-life-scott-pilgrim-1_987/index.html
[Hilo-1] (10/20) https://books.toscrape.com/catalogue/category/books/erotica_50/index.html
[Hilo-2] (11/20) https://books.toscrape.com/catalogue/category/books_1/index.html
[Hilo-1] (12/20) https://books.toscrape.com/catalogue/category/books/default_15/index.html
[Hilo-2] (13/20) https://books.toscrape.com/catalogue/category/books/humor_30/index.html
[Hilo-1] (14/20) https://books.toscrape.com/catalogue/sapiens-a-brief-history-of-humankind_996/index.html
[Hilo-2] (15/20) https://books.toscrape.com/catalogue/category/books/psychology_26/index.html
[Hilo-1] (16/20) https://books.toscrape.com/catalogue/category/books/history_32/index.html
[Hilo-2] (17/20) https://books.toscrape.com/catalogue/category/books/childrens_11/index.html
[Hilo-1] (18/20) https://books.toscrape.com/catalogue/the-dirty-little-secrets-of-getting-your-dream-job_994/index.html
[Hilo-2] (19/20) https://books.toscrape.com/catalogue/category/books/womens-fiction_9/index.html
[Hilo-1] (20/20) https://books.toscrape.com/catalogue/the-boys-in-the-boat-nine-americans-and-their-epic-quest-for-gold-at-the-1936-berlin-olympics_992/index.html
```

```
(.venv) PS C:\CrawlerWEB--ProgramacionParelalaConcurrente\CrawlerWEB--ProgramacionParelalaConcurrente> & c:\CrawlerWEB--ProgramacionParelalaConcurrente\CrawlerWEB--ProgramacionParelalaConcurrente\venv\Scripts\python.exe c:\CrawlerWEB--ProgramacionParelalaConcurrente\CrawlerWEB--ProgramacionParelalaConcurrente\benchmark.py
=====
[CONCURRENTENTE] Hilos usados      : 2
[CONCURRENTENTE] Páginas visitadas : 20
[CONCURRENTENTE] Tiempo total      : 12.35s
[CONCURRENTENTE] Tiempo por página : 0.62s
=====

[3/4] Corriendo versión CONCURRENTENTE con 4 hilo(s)...
[Hilo-1] (1/20) https://books.toscrape.com
[Hilo-1] (2/20) https://books.toscrape.com/catalogue/category/books/historical-fiction_4/index.html
[Hilo-2] (3/20) https://books.toscrape.com/catalogue/category/books/parenting_28/index.html
[Hilo-4] (5/20) https://books.toscrape.com/catalogue/category/books/politics_48/index.html
[Hilo-3] (4/20) https://books.toscrape.com/catalogue/category/books/horror_31/index.html
[Hilo-4] (6/20) https://books.toscrape.com/catalogue/category/books/romance_8/index.html
[Hilo-3] (7/20) https://books.toscrape.com/index.html
[Hilo-2] (8/20) https://books.toscrape.com/catalogue/category/books/new-adult_20/index.html
[Hilo-1] (9/20) https://books.toscrape.com/catalogue/scott-pilgrims-precious-little-life-scott-pilgrim-1_987/index.html
[Hilo-4] (10/20) https://books.toscrape.com/catalogue/category/books/erotica_50/index.html
[Hilo-2] (11/20) https://books.toscrape.com/catalogue/category/books_1/index.html
[Hilo-3] (12/20) https://books.toscrape.com/catalogue/category/books/default_15/index.html
[Hilo-1] (13/20) https://books.toscrape.com/catalogue/category/books/humor_30/index.html
[Hilo-4] (14/20) https://books.toscrape.com/catalogue/sapiens-a-brief-history-of-humankind_996/index.html
[Hilo-2] (15/20) https://books.toscrape.com/catalogue/category/books/psychology_26/index.html
[Hilo-1] (16/20) https://books.toscrape.com/catalogue/category/books/history_32/index.html
[Hilo-3] (17/20) https://books.toscrape.com/catalogue/category/books/childrens_11/index.html
[Hilo-4] (18/20) https://books.toscrape.com/catalogue/the-dirty-little-secrets-of-getting-your-dream-job_994/index.html
[Hilo-2] (19/20) https://books.toscrape.com/catalogue/category/books/womens-fiction_9/index.html
[Hilo-3] (20/20) https://books.toscrape.com/catalogue/the-boys-in-the-boat-nine-americans-and-their-epic-quest-for-gold-at-the-1936-berlin-olympics_992/index.html
```

```

=====
[CONCURRENTENTE] Hilos usados      : 4
[CONCURRENTENTE] Páginas visitadas : 20
[CONCURRENTENTE] Tiempo total      : 6.88s
[CONCURRENTENTE] Tiempo por página : 0.34s
=====

[4/4] Corriendo versión CONCURRENTENTE con 8 hilo(s)...
[Hilo-1] (1/20) https://books.toscrape.com
[Hilo-1] (2/20) https://books.toscrape.com/catalogue/category/books/historical-fiction_4/index.html
[Hilo-2] (3/20) https://books.toscrape.com/catalogue/category/books/parenting_28/index.html
[Hilo-3] (5/20) https://books.toscrape.com/catalogue/category/books/politics_48/index.html
[Hilo-4] (6/20) https://books.toscrape.com/catalogue/category/books/romance_8/index.html
[Hilo-6] (8/20) https://books.toscrape.com/catalogue/category/books/new-adult_20/index.html
[Hilo-7] (4/20) https://books.toscrape.com/catalogue/category/books/horror_31/index.html
[Hilo-8] (9/20) https://books.toscrape.com/catalogue/scott-pilgrims-precious-little-life-scott-pilgrim-1_987/index.html
[Hilo-5] (7/20) https://books.toscrape.com/index.html
[Hilo-3] (10/20) https://books.toscrape.com/catalogue/category/books/erotica_50/index.html
[Hilo-8] (11/20) https://books.toscrape.com/catalogue/category/books_1/index.html
[Hilo-1] (12/20) https://books.toscrape.com/catalogue/category/books/default_15/index.html
[Hilo-6] (13/20) https://books.toscrape.com/catalogue/category/books/humor_30/index.html
[Hilo-2] (14/20) https://books.toscrape.com/catalogue/sapiens-a-brief-history-of-humankind_996/index.html
[Hilo-5] (15/20) https://books.toscrape.com/catalogue/category/books/psychology_26/index.html
[Hilo-4] (16/20) https://books.toscrape.com/catalogue/category/books/history_32/index.html
[Hilo-7] (17/20) https://books.toscrape.com/catalogue/category/books/childrens_11/index.html
[Hilo-3] (18/20) https://books.toscrape.com/catalogue/the-dirty-little-secrets-of-getting-your-dream-job_994/index.html
[Hilo-1] (19/20) https://books.toscrape.com/catalogue/category/books/womens-fiction_9/index.html
[Hilo-5] (20/20) https://books.toscrape.com/catalogue/the-boys-in-the-boat-nine-americans-and-their-epic-quest-for-gold-at-the-1936-berlin-olympics_992/index.html

```

```

=====
[CONCURRENTENTE] Hilos usados      : 8
[CONCURRENTENTE] Páginas visitadas : 20
[CONCURRENTENTE] Tiempo total      : 5.06s
[CONCURRENTENTE] Tiempo por página : 0.25s
=====

[1/4] Corriendo versión SECUENCIAL...
[SEQ] (1/20) https://books.toscrape.com
[SEQ] (2/20) https://books.toscrape.com/catalogue/category/books/historical-fiction_4/index.html
[SEQ] (3/20) https://books.toscrape.com/catalogue/category/books/parenting_28/index.html
[SEQ] (4/20) https://books.toscrape.com/catalogue/category/books/horror_31/index.html
[SEQ] (5/20) https://books.toscrape.com/catalogue/category/books/politics_48/index.html
[SEQ] (6/20) https://books.toscrape.com/catalogue/category/books/romance_8/index.html
[SEQ] (7/20) https://books.toscrape.com/index.html
[SEQ] (8/20) https://books.toscrape.com/catalogue/category/books/new-adult_20/index.html
[SEQ] (9/20) https://books.toscrape.com/catalogue/scott-pilgrims-precious-little-life-scott-pilgrim-1_987/index.html
[SEQ] (10/20) https://books.toscrape.com/catalogue/category/books/erotica_50/index.html
[SEQ] (11/20) https://books.toscrape.com/catalogue/category/books_1/index.html
[SEQ] (12/20) https://books.toscrape.com/catalogue/category/books/default_15/index.html
[SEQ] (13/20) https://books.toscrape.com/catalogue/category/books/humor_30/index.html
[SEQ] (14/20) https://books.toscrape.com/catalogue/sapiens-a-brief-history-of-humankind_996/index.html
[SEQ] (15/20) https://books.toscrape.com/catalogue/category/books/psychology_26/index.html
[SEQ] (16/20) https://books.toscrape.com/catalogue/category/books/history_32/index.html
[SEQ] (17/20) https://books.toscrape.com/catalogue/category/books/childrens_11/index.html
[SEQ] (18/20) https://books.toscrape.com/catalogue/the-dirty-little-secrets-of-getting-your-dream-job_994/index.html
[SEQ] (19/20) https://books.toscrape.com/catalogue/category/books/womens-fiction_9/index.html
[SEQ] (20/20) https://books.toscrape.com/catalogue/the-boys-in-the-boat-nine-americans-and-their-epic-quest-for-gold-at-the-1936-berlin-olympics_992/index.html

```

Evaluación del desempeño

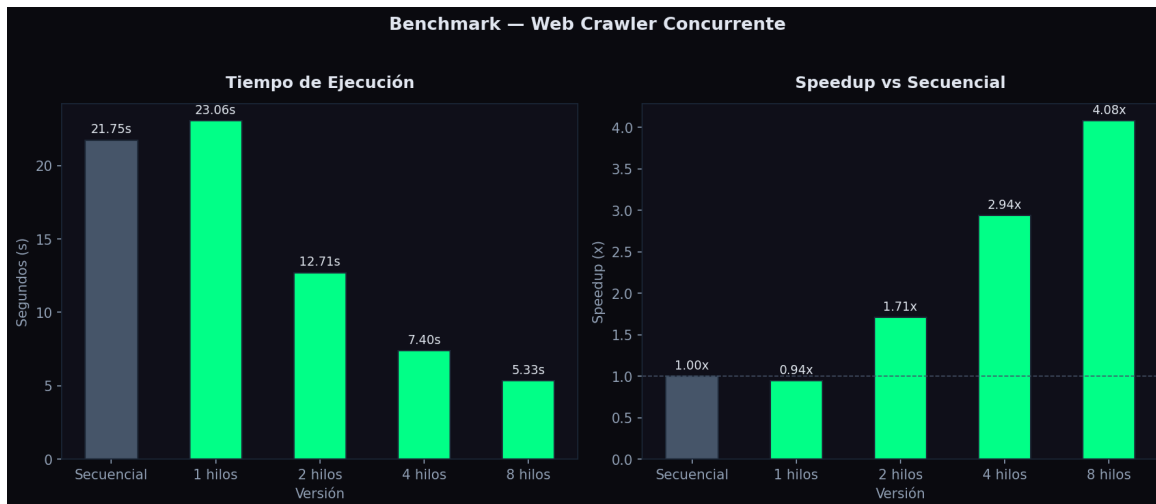
Para cuantificar la mejora en el rendimiento, utilizamos la métrica de Speedup (Aceleración), la cual se define como la relación entre el tiempo del programa secuencial y el tiempo del programa concurrente:

$$\text{Speedup} = \text{Tiempo de la versión secuencial} / \text{Tiempo de la versión paralela}$$

A continuación, se presentan los tiempos obtenidos al ejecutar el algoritmo de forma tradicional (secuencial) y al paralelizarlo variando la cantidad de hilos de trabajo (1, 2, 4 y 8 hilos):

```

=====
Versión                Tiempo (s)    Speedup
=====
Secuencial             21.75       1.00x
Paralela (1 hilos)     23.06       0.94x
Paralela (2 hilos)     12.71       1.71x
Paralela (4 hilos)     7.40        2.94x
Paralela (8 hilos)     5.33        4.08x
=====
  
```



Versión	Tiempo en segundos	Speedup
Secuencial	21.75 s	1.00x
Concurrente (1 hilo)	23.06 s	0.94x

Concurrente (2 hilos)	12.71 s	1.71x
Concurrente (4 hilos)	7.40 s	2.94x
Concurrente (8 hilos)	5.33 s	4.08x

Al analizar detalladamente la tabla comparativa y las gráficas generadas por nuestras pruebas, pudimos identificar comportamientos muy interesantes sobre cómo responde verdaderamente la arquitectura del sistema. Un detalle que destaca inmediatamente es el costo inicial de la concurrencia. Resulta curioso observar que la versión concurrente configurada con un solo hilo tardó 23.06 segundos, siendo ligeramente más lenta que la versión puramente secuencial, la cual tomó 21.75 segundos. Precisamente esto nos sirvió para evidenciar el overhead o sobrecarga administrativa del sistema. Este tiempo extra se debe a la inicialización de la estructura de la cola, la creación del hilo de trabajo y, sobre todo, a los microsegundos que el procesador invierte adquiriendo y liberando constantemente los candados (Locks) para acceder a las variables compartidas, un costo del cual la versión secuencial está completamente libre.

Sin embargo, notamos que a partir de la implementación de dos hilos, el beneficio de superponer las tareas supera rápidamente este costo inicial. Nuestras gráficas demuestran una escalabilidad notable conforme incrementamos la cantidad de workers. Al utilizar 8 hilos simultáneos, el tiempo de ejecución se redujo drásticamente a tan solo 5.33 segundos, logrando una aceleración o Speedup de 4.08x. Esto nos comprueba de forma empírica que nuestra arquitectura logró enmascarar con éxito los tiempos muertos de la red, permitiendo que el sistema procese y descubra nuevas URLs mientras otros hilos todavía están bloqueados esperando la respuesta del servidor web.

Finalmente, al evaluar estos números también comprendimos la razón por la cual el rendimiento no crece de manera perfectamente proporcional; es decir, por qué usar 8 hilos no nos otorgó un Speedup exacto de 8.0x. Este fenómeno nos ilustró a la perfección la aplicación real de la Ley de Amdahl, la cual dicta que la mejora de un sistema paralelo siempre estará limitada por su porción estrictamente secuencial. En nuestro proyecto, esta limitación física ocurre por la fila que se hace cuando varios hilos intentan bloquear el Mutex (`visited_lock`) al mismo tiempo, sumado al esfuerzo de la CPU al momento de utilizar la librería BeautifulSoup para procesar el texto HTML. A pesar de esto, lograr reducir casi el 75% del tiempo total de ejecución justifica plenamente nuestro diseño y demuestra que la concurrencia es la estrategia correcta para optimizar problemas limitados por entrada y salida.

Conclusiones

Durante el desarrollo y la culminación de este proyecto, logramos consolidar nuestro conocimiento teórico sobre la programación concurrente al aplicarlo a un entorno real fuertemente limitado por las operaciones de red. Uno de los aprendizajes más significativos fue comprender en la práctica la necesidad crítica de utilizar mecanismos de sincronización cuando se trabaja con memoria compartida. Al implementar bloqueos mediante “threading.Lock”, logramos evitar la corrupción de datos y las temidas condiciones de carrera que surgían cuando varios hilos intentaban registrar o leer páginas visitadas de manera simultánea. Asimismo, esta práctica nos permitió observar de primera mano el impacto del overhead en la computadora, comprobando que la gestión de hilos, colas y candados conlleva un costo de procesamiento inicial que solo se compensa cuando el sistema superpone suficientes tareas.

Pensando en el futuro y en las áreas de mejora, si tuviéramos que llevar este sistema a un entorno de producción a gran escala, realizaríamos varias evoluciones arquitectónicas. Sabemos que la librería threading nos brindó resultados excelentes para este caso de estudio, pero estamos conscientes de las limitaciones que impone el GIL (Global Interpreter Lock) en Python. Por ello, consideramos que una mejora importante sería reestructurar el código hacia un modelo de concurrencia asíncrona, lo que nos permitiría manejar miles de peticiones simultáneas sin generar sobrecarga en el sistema operativo por el cambio de contexto. Además, para escalar el rastreo web más allá de los límites físicos de una sola máquina, sería ideal sustituir nuestra cola gestionada en la memoria RAM por un sistema de colas de tareas distribuido. A la par de esto, resulta indispensable integrar un sistema gestor de bases de datos persistente, para garantizar que la información extraída se almacene de forma segura en disco y no se pierda al terminar la ejecución del programa.

En pocas palabras, la aplicación de la concurrencia le aportó una viabilidad absoluta y una eficiencia extrema a nuestro sistema. A lo largo del proyecto quedó claro que, frente a un cuello de botella causado por la latencia de la red, un enfoque estrictamente secuencial resulta inescalable, ya que mantiene al procesador inactivo desperdiciando recursos. Al paralelizar el trabajo con nuestra arquitectura, logramos enmascarar esos tiempos muertos y transformamos un algoritmo lineal y bloqueante en una herramienta verdaderamente rápida y optimizada. El haber logrado reducir el tiempo de ejecución de casi 22 segundos a poco más de 5 segundos, obteniendo un Speedup de 4.08x, nos demostró que aplicar técnicas concurrentes no es solo un lujo matemático, sino un requisito indispensable para desarrollar software moderno que sea capaz de interactuar eficientemente con la web.

Referencias

- *Tanenbaum, A. S., & Bos, H. (2015). Modern Operating Systems (4ta ed.). Pearson.*
- *Python Software Foundation. (2023). threading — Thread-based parallelism. Documentación oficial de Python 3. Recuperado de <https://docs.python.org/3/library/threading.html>*
- *Python Software Foundation. (2023). queue — A synchronized queue class. Documentación oficial de Python 3. Recuperado de <https://docs.python.org/3/library/queue.html>*